

微光招新 C-EASY-2 准备

C-EASY-2 Step 1：指针与结构体

第一部分

(1)指针：指针就是存地址的变量。int *p = &a; 把 a 的地址给 p，*p 就能直接改 a 的值。64 位系统里指针固定占 8 字节，32位占4个字节。指针大小与系统位数有关，与指向的数据类型无关。

(2)见pointer_quiz.c

```
#include <stdio.h>
```

```
int main() { int x = 10; int* p = &x; *p = 20;
```

```
    int arr[3] = {3, 6, 9};  
    int *q = arr;  
    int y = ++*arr + *++q;  
  
    printf("%d %d\n", x, y);  
    return 0;
```

```
}
```

输出结果：20，10 原因:1.执行p=20，x的值变为20。++arr先取值后自增，结果为4，++q即为arr[1]结果为6，和为10

(3)野指针：没初始化的指针，指向未知内存，写它程序会崩。定义时就给初值，或者先置 NULL。

(4)见swap.intro.c,通过swap(&x,&y)调用

```
void swap(int a,intb) { int temp=*a; *a=*b; *b=temp; }
```

第二部分

(1)

```
typedef struct PerInfo
```

```
{
```

```
    char name[10];  
  
    char sex;  
  
    int age;
```

```
double height;
```

```
}M;
```

(2)结构体指针就是指向结构体的指针，访问成员用->而不是.，比如p->age。

(3)内存对齐计算过程：PerInfo 结构体实际占用 24 字节（不是 $10+1+4+8=23$ ）。

计算过程：

1. name[10] 占 0-9 位置。
2. sex 占 10 位置。
3. age 是 int（4字节），起始地址必须是 4 的倍数，所以地址 11 必须留空，从 12 开始，占 12-15。
4. height 是 double（8字节），起始地址必须是 8 的倍数，从 16 开始，占 16-23。
5. 总大小必须是最大成员（double 8字节）的倍数，24 是 8 的倍数，所以最终大小是 24 字节。

(4) typedef struct PerInfo

```
{ char name[10];
```

```
double height;
```

```
char sex;
```

```
int age;
```

```
}M;
```

调整顺序变为32个字节。原因：name[10] 占 0-9；double height 必须从 8 的倍数地址开始，地址 10-15 被迫留空，height 占 16-23；char sex 占 24；int age 必须从 4 的倍数地址开始，地址 25-27 留空，age 占 28-31。总大小为 32 字节。

Step 2：链操作

1.链表：链表是动态分配，用malloc申请内存，再存储数据，大小理论无上限，可随时增减，每创建一个节点会存一个next指针，消耗内存。节点分散在各处，靠指针相连。访问效率低，必需遍历。插入删除效率高于数组。

数组：静态分配，先定好大小再存储，大小固定，内存连续。可随机访问，效率高。插入删除会移动所有元素，效率低，内存无额外开销。

2.单向链表节点由两部分组成：数据域和指针域。数据域存储数据，指针域存储下一节点的地址并连接节点，只能单向遍历。

定义一个只存储一个整数的单向链表节点：

```
typedef struct Node
```

```
{
```

```
    int data;  
  
    struct Node *next;
```

```
}Node;
```

设计create_node函数:

```
Node* create_node(int value)
```

```
{
```

```
    Node* p=(Node*)malloc(sizeof(Node));  
  
    if (p==NULL)  
    {  
        return NULL;  
    }  
  
    p->data=value;  
  
    p->next=NULL;  
  
    return p;
```

```
}
```

头插函数:

```
void insert_head(Node** head, int value)
```

```
{
```

```
    Node* p = create_node(value);  
  
    if (p == NULL)  
    {  
        return;  
    }  
  
    *head = p;
```

```
p->next = *head;

*head = p;
```

```
}
```

尾插函数：

```
void insert_tail(Node**head,int value)
```

```
{
```

```
    Node*p=create_node(value);

    if (p==NULL)

        return;

    else if(*head==NULL)

    {

        *head=p;

        return;

    }

    else

    {

        Node*current=*head;

        while(current->next!=NULL)

        {

            current=current->next;

        }

        current->next = p;

    }
```

```
}
```

打印：

```
void printf_list(Node*head)
```

```
{
```

```
Node*current=head;

while(current!=NULL)

{

    printf("%d->",current->data);

    current=current->next;

}

printf("NULL\n");
```

```
}
```

查找:

```
int find_node(Node*head,int a)
```

```
{
```

```
Node*current=head;

int b=1;

while(current!=NULL)

{

    if(current->data==a)

    {

        return b;

    }

    else

    {

        b++;

        current=current->next;

    }

}
```

```
return -1;
```

```
}
```

删除:

```
int delete_node(Node**head,int n)
```

```
{
```

```
    if(*head==NULL || n<1)
    {
        return 0;
    }
    if(n==1)
    {
        Node*temp=*head;
        *head=(*head)->next;
        free(temp);
        return 1;
    }
    Node*a=*head;
    Node*b=(*head)->next;
    int x=2;
    while(b!=NULL&& x<n)
    {
        a=b;
        b=b->next;
        x++;
    }
    if(b==NULL)
```

```
{  
    return 0;  
}  
  
a->next=b->next;  
  
free(b);  
  
return 1;
```

```
}
```

修改:

```
int update_node(Node*head,int old_val,int new_val)
```

```
{
```

```
Node*current=head;  
  
while(current->data!=old_val&&current!=NULL)  
  
{  
    current=current->next;  
}  
  
if (current!=NULL)  
  
{  
    current->data=new_val;  
  
    return 1;  
}  
  
else  
  
{  
    return 0;  
}
```

```
}
```

反转：

`Nodereverse_list(Nodehead)`

{

```
Node*prev=NULL;
```

```
Node*current=head;
```

```
Node*next=NULL;
```

```
while(current!=NULL)
```

```
{
```

```
    next=current->next;
```

```
    current->next=prev;
```

```
    prev=current;
```

```
    current=next;
```

```
}
```

```
return prev;
```

}

Day 4：数组与函数基础

今日完成

- 独立完成了三个基础练习：
 1. `Day4_array_func.c`：输入 5 个数，遍历找出最大值。
 2. `Day4_array_func.c`：将找最大值的逻辑封装成函数。
 3. `Day4_array_func.c`：实现了数组逆序函数。

踩坑记录（重点）

- **数组传参认知**：函数接收数组时，`int arr[]` 是指接收数组首地址；同时必须传一个 `int size`，否则函数不知道数组有多长。
- **for 循环语法**：三个部分必须用分号隔开，不能写成逗号（如 `i=0, j=size-1; i<j; i++, j--` 才是对的）。

- **数组下标**：下标必须从 0 开始遍历，写成 `i=1; i<6` 不仅会漏掉第一个数，还会导致越界错误。

Day 6: 指针入门

1. 指针基础

- `&a`：取变量 `a` 的地址。
- `int *p = &a;`：定义指针变量 `p`，把 `a` 的地址存进去。
- `*p`：解引用，顺着 `p` 里的地址找到 `a`，可以读也可以改。
- 验证结果：`&a` 和 `p` 打印出来的地址完全相同；`*p = 20` 之后 `a` 真的变成了 20。

2. 指针版 swap

- 值传递：`void swap(int a, int b)` 收到的只是实参的“复印件”，改复印件不影响原件。
- 地址传递：`void swap(int *a, int *b)` 收到的是变量的地址，通过 `*a` 和 `*b` 直接操作原内存，才能真正交换。
- 调用：`swap(&x, &y);`

3. 数组指针遍历

- 数组名就是首地址，所以 `int *p = arr;` 不需要加 `&`。
- `*p` 取当前元素，`p++` 让指针往后移一格。
- 用指针遍历和用 `arr[i]` 效果完全等价。

4. 指针的大小

- 在 64 位系统里，指针变量占 8 个字节（因为地址是 64 位）。
- 指针的大小与它指向的类型无关，只与系统位数有关。